

DOCUMENTACIÓN TÉCNICA

Gestor de Tareas

Aplicación Web Full Stack

Gonzalo Iván Pedraza

Full Stack Developer
Tucumán, Argentina — 2026

1. Descripción del Proyecto

El Gestor de Tareas es una aplicación web full stack diseñada para la gestión de tareas personales. Permite a los usuarios organizar su trabajo diario con funcionalidades avanzadas como subtareas, notificaciones por fecha límite, drag & drop para reordenar, búsqueda en tiempo real y modo oscuro.

El proyecto fue desarrollado con tecnologías modernas (React, Node.js, TypeScript, MongoDB) como parte del proceso de migración del Sistema de Gestión Comercial a un stack tecnológico actualizado.

Características Principales

- Autenticación completa con JWT — registro, login y logout seguros
- CRUD completo de tareas con validaciones en frontend y backend
- Subtareas anidadas con barra de progreso visual
- Notificaciones por fecha límite — alertas para tareas vencidas, que vencen hoy o mañana
- Búsqueda en tiempo real — filtra por título, descripción y categoría
- Filtros por estado (pendiente/completada) y prioridad (baja/media/alta)
- Estadísticas visuales en tiempo real
- Dark mode con persistencia en localStorage
- Drag & drop para reordenar tareas

2. Stack Tecnológico

Frontend

Tecnología	Versión	Uso
React	18	Biblioteca de UI
TypeScript	5	Tipado estático
Tailwind CSS	4	Framework de estilos
Vite	7	Bundler y dev server
@dnd-kit	latest	Drag & drop
@headlessui/react	latest	Componentes accesibles (modales)
date-fns	latest	Manejo de fechas
lucide-react	latest	Librería de íconos
axios	latest	Cliente HTTP

Backend

Tecnología	Versión	Uso
Node.js	18+	Runtime de JavaScript
Express	4	Framework HTTP
TypeScript	5	Tipado estático
MongoDB	6	Base de datos NoSQL
Mongoose	8	ODM para MongoDB
JWT	latest	Autenticación por tokens
bcryptjs	latest	Hash de contraseñas

3. Estructura del Proyecto

Frontend — src/

Carpeta / Archivo	Descripción
components/TaskCard.tsx	Tarjeta individual de tarea con acciones
components/TaskForm.tsx	Modal para crear y editar tareas
components/TaskFilters.tsx	Panel de filtros por estado y prioridad
components/TaskStats.tsx	Cards de estadísticas en tiempo real
components/TaskSearch.tsx	Barra de búsqueda en tiempo real
components/DueDateBanner.tsx	Banners de alerta por fecha límite
components/SortableTaskCard.tsx	Wrapper drag & drop para TaskCard
contexts/AuthContext.tsx	Estado global de autenticación
contexts/TaskContext.tsx	Estado global de tareas con CRUD
contexts/ThemeContext.tsx	Estado global del tema claro/oscuro
hooks/useNotifications.ts	Hook para notificaciones del sistema
pages/LoginPage.tsx	Página de inicio de sesión
pages/RegisterPage.tsx	Página de registro
pages/DashboardPage.tsx	Dashboard principal con todas las funciones
services/api.ts	Instancia configurada de axios
services/taskService.ts	Métodos para llamadas a la API de tareas
types/index.ts	Interfaces TypeScript del proyecto

Backend — src/

Carpeta / Archivo	Descripción
controllers/authController.ts	Lógica de registro, login y perfil
controllers/taskController.ts	CRUD de tareas y subtareas
middleware/auth.ts	Validación de JWT en rutas protegidas
models/User.ts	Schema de usuario en MongoDB
models/Task.ts	Schema de tarea con subtareas embebidas
routes/auth.ts	Rutas de autenticación
routes/tasks.ts	Rutas de tareas y subtareas

4. Endpoints de la API

Autenticación — /api/auth

Método	Endpoint	Descripción	Auth
POST	/api/auth/register	Registrar nuevo usuario	No
POST	/api/auth/login	Iniciar sesión	No
GET	/api/auth/me	Obtener usuario actual	Sí

Tareas — /api/tasks

Método	Endpoint	Descripción	Auth
GET	/api/tasks	Obtener todas las tareas (con filtros)	Sí
GET	/api/tasks/:id	Obtener tarea por ID	Sí
POST	/api/tasks	Crear nueva tarea	Sí
PUT	/api/tasks/:id	Actualizar tarea	Sí
DELETE	/api/tasks/:id	Eliminar tarea	Sí
GET	/api/tasks/stats	Obtener estadísticas del usuario	Sí

Subtareas — /api/tasks/:id/subtasks

Método	Endpoint	Descripción	Auth
POST	/api/tasks/:id/subtasks	Agregar subtarea	Sí
PUT	/api/tasks/:id/subtasks/:subtaskId	Toggle completar subtarea	Sí
DELETE	/api/tasks/:id/subtasks/:subtaskId	Eliminar subtarea	Sí

5. Instalación y Configuración

Prerrequisitos

- Node.js 18 o superior
- MongoDB instalado localmente o cuenta en MongoDB Atlas
- npm o yarn

Variables de Entorno — Backend (.env)

Variable	Descripción	Ejemplo
PORT	Puerto del servidor	5000
MONGODB_URI	Cadena de conexión a MongoDB	mongodb://localhost:27017/gestor-tareas
JWT_SECRET	Clave secreta para JWT	mi_clave_secreta
JWT_EXPIRES_IN	Expiración del token	7d

Variables de Entorno — Frontend (.env)

Variable	Descripción	Ejemplo
VITE_API_URL	URL base de la API	http://localhost:5000/api

Pasos de Instalación

1. Clonar el repositorio:
`git clone https://github.com/Gonza28fq/gestor-de-tareas.git`
2. Instalar dependencias del backend:
`cd backend && npm install && npm run dev`
3. Instalar dependencias del frontend:
`cd frontend && npm install && npm run dev`
4. Abrir el navegador en `http://localhost:5173`

6. Autor

Campo	Detalle
Nombre	Gonzalo Iván Pedraza
Rol	Full Stack Developer
Ubicación	San Miguel de Tucumán, Argentina
Email	gonza280797@hotmail.com
Portfolio	https://gonza28fq.github.io
GitHub	https://github.com/Gonza28fq
LinkedIn	https://www.linkedin.com/in/gonzalo-pedraza-b8b568298